

INTERVIEW PREPARATION GUIDE

Node.js Interview Questions & Answers

A complete, structured reference covering core concepts, the event loop, streams, Express & REST APIs, security, databases, testing, performance, microservices, architecture, and hands-on coding challenges — for Junior to Senior level interviews.

113

QUESTIONS

15

TOPIC AREAS

40+

CODE EXAMPLES

Compiled reference · August 2026

Table of Contents

1. Node.js Fundamentals & Architecture	Q1–10	2. Modules & Package Management	Q11–18
3. Async Programming & the Event Loop	Q19–28	4. Streams & Buffers	Q29–36
5. File System & OS	Q37–41	6. Error Handling	Q42–47
7. Process, Child Processes, Cluster & Workers	Q48–54	8. Express.js, Middleware & REST APIs	Q55–64
9. Security & Authentication	Q65–72	10. Database Integration & ORMs	Q73–78
11. Testing & Debugging	Q79–84	12. Performance, Memory & Scaling	Q85–92
13. Microservices & Real-time	Q93–98	14. Design Patterns & Architecture	Q99–103
15. Practical Coding Challenges	Q104–113		

1 Node.js Fundamentals & Architecture

Core concepts every candidate is expected to know cold.

Q1. What is Node.js, and why is it used?

BEGINNER

Node.js is a cross-platform, open-source JavaScript **runtime** built on Chrome's V8 engine. It lets JavaScript run outside the browser, on the server, with access to the file system, network, and OS. It is not a framework or a language — it's an execution environment.

It's popular for backend development because it:

- Uses non-blocking, event-driven I/O, which makes it efficient for I/O-heavy workloads (APIs, real-time apps, streaming).
- Lets teams share one language (JavaScript/TypeScript) across frontend and backend.
- Has a massive package ecosystem via npm.
- Handles many concurrent connections with a small memory footprint compared to thread-per-request models.

Q2. Is Node.js single-threaded? Explain the actual threading model.

INTERMEDIATE

Partially — and this is a very common trick question. **Your JavaScript code** runs on a single thread (the main event loop thread). However, Node.js itself is not single-threaded:

- **libuv** maintains a thread pool (default size 4) used for expensive operations that don't have native async OS support — e.g. file system access, DNS lookups (`dns.lookup`), certain crypto functions (`pbkdf2`, `scrypt`), and compression.
- Network I/O (sockets) uses the OS's native async mechanisms (epoll/kqueue/IOCP) and doesn't need the thread pool at all.
- Since Node 10.5, **Worker Threads** let you run actual parallel JS on separate threads for CPU-bound work.

So: single-threaded JS execution, but a multi-threaded runtime underneath.

Q3. What is the V8 engine and what role does it play in Node.js?

BEGINNER

V8 is Google's open-source JavaScript/WebAssembly engine (also used in Chrome). It compiles JavaScript directly to native machine code using JIT (Just-In-Time) compilation rather than interpreting bytecode, which makes execution fast. Node.js embeds V8 to execute JavaScript and extends it with C++ bindings that expose OS-level capabilities (file system, networking, etc.) that aren't available in a browser sandbox.

Q4. What is libuv, and why does Node.js need it?

INTERMEDIATE

libuv is a C library that provides Node.js with its **event loop**, asynchronous I/O, the thread pool, and cross-platform abstractions for networking, file system access, timers, and child processes. V8 only executes JavaScript — it has no concept of I/O or timers. libuv is what bridges that gap and gives Node.js its non-blocking behavior on Windows, Linux, and macOS despite each OS having very different native async I/O APIs.

Q5. Describe the overall Node.js runtime architecture.

ADVANCED

Roughly, from top to bottom:

- **Your JS code** — application/business logic.
- **Node.js core/standard library** — `fs`, `http`, `events`, etc. (partly JS, partly C++ bindings).
- **V8** — compiles and executes JavaScript, manages the heap and garbage collection.
- **libuv** — event loop, thread pool, async I/O, timers.
- **OS** — actual system calls (epoll on Linux, kqueue on macOS, IOCP on Windows).

Understanding this stack explains why Node scales well for I/O-bound work but struggles with CPU-bound work on the main thread.

Q6. Key differences between Node.js and browser JavaScript.

BEGINNER

- No `window`, `document`, or DOM in Node — instead there's `global`, `process`, `Buffer`.
- Node supports CommonJS (`require`) natively and ESM; browsers only support ESM (`import`) natively.
- Node has file system, OS, and process-level APIs; browsers sandbox all of that away for security.
- Browser JS is subject to same-origin policy/CORS; Node makes outbound requests freely.

Q7.

How does Node's concurrency model compare to a traditional thread-per-request server (e.g. classic Apache/PHP or Java servlets)?

INTERMEDIATE

Thread-per-request models spin up (or borrow from a pool) an OS thread per incoming request. Threads are expensive — each has its own stack (often 1MB+), and context switching between many threads adds CPU overhead. This model scales well for CPU-heavy work but poorly for thousands of concurrent, mostly-idle I/O-bound connections.

Node uses one thread to handle many connections concurrently via an event loop — while one request waits on I/O (DB query, disk read, API call), the thread is free to process other requests. This makes Node very memory-efficient for I/O-bound workloads like APIs and chat servers, but a single CPU-bound request (heavy computation) blocks everyone else until it finishes.

Q8. What are the important global objects in Node.js? BEGINNER

- `global` — the global namespace object (like `window` in browsers).
- `process` — info and control over the current Node process (env vars, argv, exit codes, signals).
- `__dirname` / `__filename` — absolute path of the current module's directory/file (CommonJS only).
- `Buffer` — for handling raw binary data.
- `setTimeout`, `setInterval`, `setImmediate`, `queueMicrotask` — scheduling.
- `module`, `exports`, `require` — module system (CommonJS).

Q9. What is the Node.js REPL? BEGINNER

REPL stands for **Read-Eval-Print-Loop**. Running `node` with no arguments opens an interactive shell that reads a JS expression, evaluates it, prints the result, and loops. It's useful for quickly testing snippets, inspecting APIs, or debugging without creating a file.

Q10. What are the main advantages and disadvantages of Node.js?

INTERMEDIATE

Advantages: fast for I/O-bound workloads, single language across the stack, huge npm ecosystem, great for real-time apps (chat, streaming, collaboration tools), lightweight and fast to start.

Disadvantages: not ideal for CPU-heavy tasks (image/video processing, heavy computation) unless offloaded to worker threads or another service; callback/async code can get complex without discipline; being single-threaded for JS execution means one unhandled long-running synchronous operation blocks all requests; the huge npm ecosystem means variable code quality and larger supply-chain surface area.

2 Modules & Package Management

How Node organizes code and manages dependencies.

Q11. What is a module in Node.js? Explain CommonJS vs ES Modules.

BEGINNER

A module is a reusable, self-contained unit of code — every file in Node is treated as its own module with its own scope. Node supports two module systems:

- **CommonJS (CJS)** — the original Node system. Uses `require()` to import and `module.exports` / `exports` to export. Loading is synchronous and can happen conditionally/dynamically anywhere in the code.
- **ES Modules (ESM)** — the standard JS module system, using `import` / `export`. Enabled by setting `"type": "module"` in package.json or using the `.mjs` extension. Loading is asynchronous under the hood and statically analyzable (imports must be top-level), which enables tree-shaking.

Q12. require vs import — what are the practical differences?

INTERMEDIATE

- `require()` is synchronous and can be called conditionally/anywhere; `import` is hoisted, static, and must appear at the top level (except dynamic `import()`, which returns a Promise).
- CJS exports are a mutable object copy; ESM exports are live bindings (read-only references that update if the source changes).
- CJS has no built-in top-level `await` (pre-Node 22 workarounds needed); ESM supports top-level `await` natively.
- Interop: ESM can import CJS modules (as a default export), but CJS cannot directly `require()` an ESM module — you'd need dynamic `import()`.

Q13.**What happens when you `require()` the same module multiple times?****INTERMEDIATE**

Node **caches** modules by resolved file path after the first `require()`. Subsequent requires return the same cached exports object instead of re-executing the file. This means module-level state is shared across every file that requires it (singletons work naturally), and it's also why mutating an object exported from a module affects every other file using it. The cache can be inspected/cleared via `require.cache`.

Q14. Difference between `module.exports` and `exports`. **INTERMEDIATE**

`exports` is just a variable that initially points to the same object as `module.exports`. You can add properties to `exports` (e.g. `exports.foo = ...`) and it works because both point to the same object. But if you **reassign** `exports = something`, it breaks the link — only reassigning `module.exports` actually changes what the module returns, since that's what `require()` actually reads.

JAVASCRIPT

```
// Works: mutating the shared object
exports.add = (a, b) => a + b;

// Breaks the link to module.exports - require() won't see this
exports = { add: (a, b) => a + b };

// Correct way to export a single value/function
module.exports = function add(a, b) {
  return a + b;
};
```


Q15. What is package.json and what do its key fields mean?

BEGINNER

package.json is the manifest file describing a Node project — its metadata, dependencies, and scripts. Important fields:

- `main` — entry file when the package is required.
- `type` — `"module"` for ESM or `"commonjs"` (default).
- `scripts` — named CLI commands runnable via `npm run`.
- `engines` — the Node/npm versions the project supports.
- `dependencies` / `devDependencies` / `peerDependencies` — see Q16.
- `version` — semantic version of the package itself.

Q16.

Difference between dependencies, devDependencies, peerDependencies, and optionalDependencies.

INTERMEDIATE

- **dependencies** — required at runtime in production (e.g. express, mongoose).
- **devDependencies** — only needed during development/build/test (e.g. jest, eslint, nodemon); not installed in a `--production` install.
- **peerDependencies** — declares that your package expects the *consumer's* project to provide a compatible version (common in plugins, e.g. a React component library expecting the host app's React).
- **optionalDependencies** — installed if possible, but the package should not crash if installation fails (e.g. an optional native binding).

Q17.**Explain semantic versioning and the purpose of package-lock.json.****INTERMEDIATE**

Semver format is **MAJOR.MINOR.PATCH** (e.g. 4.18.2): MAJOR = breaking changes, MINOR = backward-compatible features, PATCH = backward-compatible bug fixes.

- `^4.18.2` — allow MINOR and PATCH updates (up to, not including, 5.0.0).
- `~4.18.2` — allow only PATCH updates (up to, not including, 4.19.0).
- Exact `4.18.2` — pin to that exact version.

`package-lock.json` records the exact resolved version (and dependency tree) that was installed, so every machine/CI run gets an identical `node_modules` tree regardless of semver ranges in package.json — this guarantees reproducible builds.

Q18. npm vs yarn vs pnpm — what actually differs?**INTERMEDIATE**

- **npm** — default, bundled with Node. Modern versions (v7+) support workspaces and have a fast, reliable resolver.
- **yarn** — historically faster and offered deterministic lockfiles before npm did; Yarn Berry (v2+) supports Plug'n'Play (no `node_modules` at all).
- **pnpm** — uses a global content-addressable store and symlinks/hardlinks packages into `node_modules`, saving significant disk space and installing faster across projects; it's also stricter about phantom dependencies (a package can't accidentally use a dependency it didn't declare).

Functionally they all read/write package.json and resolve a dependency graph — the differences are speed, disk usage, and strictness.

3 Async Programming & the Event Loop

The topic that shows up in almost every Node.js interview.

Q19. Explain the Node.js event loop and its phases. ADVANCED

The event loop is what lets Node perform non-blocking I/O despite JS being single-threaded — it continuously checks for completed async work and runs the associated callbacks. Each iteration ("tick") passes through six phases, in order:

- **Timers** — runs callbacks scheduled by `setTimeout` / `setInterval` whose threshold has elapsed.
- **Pending callbacks** — executes I/O callbacks deferred from the previous cycle (e.g. certain TCP errors).
- **Idle, prepare** — internal use only.
- **Poll** — retrieves new I/O events and executes their callbacks; this is where most work happens, and the loop can block here waiting for events if nothing else is scheduled.
- **Check** — runs `setImmediate` callbacks, right after the poll phase.
- **Close callbacks** — e.g. `socket.on('close', ...)`.

Between **every** phase transition (and every callback), Node fully drains the microtask queue (Promises, `process.nextTick`).

Q20.**Difference between `process.nextTick()`, Promise microtasks, `setImmediate`, and `setTimeout(fn, 0)`.****ADVANCED**

- `process.nextTick()` — highest priority; runs after the current operation completes, before any other microtask or I/O, even before Promise callbacks.
- **Promise callbacks** (`.then` /microtasks) — run right after `nextTick` queue is drained, still before the event loop continues.
- `setImmediate()` — runs in the **check** phase, i.e. after the poll phase of the current loop iteration.
- `setTimeout(fn, 0)` — runs in the **timers** phase; timing relative to `setImmediate` is not guaranteed at the top level (OS timer resolution), but inside an I/O callback, `setImmediate` always fires before `setTimeout(fn, 0)`.

JAVASCRIPT

```
console.log('start');

setTimeout(() => console.log('timeout'), 0);
setImmediate(() => console.log('immediate'));

Promise.resolve().then(() => console.log('promise'));
process.nextTick(() => console.log('nextTick'));

console.log('end');
```

Typical output: `start, end, nextTick, promise, timeout/immediate (order varies), immediate/timeout` — but inside I/O callbacks, immediate always wins over timeout.

Q21. What is blocking vs non-blocking code? Give an example.

BEGINNER

Blocking code halts further JS execution until the current operation finishes (e.g. `fs.readFileSync`). **Non-blocking** code kicks off an operation and immediately returns control, invoking a callback/Promise once it's done (e.g. `fs.readFile`), letting the event loop serve other work meanwhile.

JAVASCRIPT

```
const fs = require('fs');

// Blocking: nothing else runs until the file is fully read
const data = fs.readFileSync('big-file.txt', 'utf8');
console.log(data);

// Non-blocking: control returns immediately, callback runs later
fs.readFile('big-file.txt', 'utf8', (err, data) => {
  if (err) throw err;
  console.log(data);
});
console.log('This logs before the file content, in the async version')
```

Q22. What is "callback hell" and how do you avoid it? BEGINNER

Callback hell is deeply nested callbacks (a "pyramid of doom") that result from chaining multiple async operations with plain callbacks, making code hard to read, debug, and error-handle. It's avoided by:

- Converting callback-based APIs to Promises (`util.promisify`) and chaining with `.then()`.
- Using `async` / `await` for linear, synchronous-looking code.
- Splitting logic into small, named functions instead of anonymous inline callbacks.
- Using control-flow libraries (historically `async.js`) when needed.

Q23.**Explain Promises: states, chaining, and the combinator methods.****INTERMEDIATE**

A Promise represents an eventual result of an async operation. It has three states: **pending**, **fulfilled**, or **rejected** — once settled (fulfilled/rejected) it can never change state again. Combinator methods:

- `Promise.all(promises)` — resolves when all resolve, with an array of results; rejects immediately if any one rejects.
- `Promise.allSettled(promises)` — always resolves, giving the status (fulfilled/rejected) of every promise — useful when you need every result regardless of failures.
- `Promise.race(promises)` — settles as soon as the first promise settles (fulfilled or rejected).
- `Promise.any(promises)` — resolves as soon as the first one fulfills; rejects only if all reject.

Q24.**How does async/await work under the hood, and how do you handle errors with it?****INTERMEDIATE**

`async` / `await` is syntactic sugar over Promises. An `async` function always returns a Promise; `await` pauses execution of that function (without blocking the event loop) until the awaited Promise settles, then either returns the resolved value or throws the rejection as a catchable exception. Errors are handled with plain `try/catch`, which makes async error handling read like synchronous code.

JAVASCRIPT

```
async function getUser(id) {  
  try {  
    const res = await fetch(`/api/users/${id}`);  
    if (!res.ok) throw new Error(`HTTP ${res.status}`);  
    return await res.json();  
  } catch (err) {  
    console.error('Failed to fetch user:', err.message);  
    throw err; // re-throw so the caller can also react  
  }  
}
```

Q25. What is an EventEmitter, and when would you use one?

INTERMEDIATE

`EventEmitter` (from the `events` module) implements the observer pattern: objects can emit named events, and other code can subscribe to react to them. Much of Node's core (streams, HTTP servers, process) is built on it. It's useful for decoupling producers and consumers of an event inside a single process.

JAVASCRIPT

```
const { EventEmitter } = require('events');

class OrderService extends EventEmitter {
  placeOrder(order) {
    // ... business logic
    this.emit('order:placed', order);
  }
}

const orders = new OrderService();
orders.on('order:placed', (order) => {
  console.log('Send confirmation email for', order.id);
});
orders.placeOrder({ id: 42 });
```

Q26.

Difference between synchronous and asynchronous versions of fs methods.

BEGINNER

`fs.readFileSync()` blocks the event loop until the read finishes and returns the data directly (or throws). `fs.readFile()` is non-blocking — it returns immediately and delivers the result via a callback (or Promise via `fs.promises` / `fs/promises`) once the underlying libuv thread-pool operation completes. In a server handling concurrent requests, sync methods should be avoided on the hot path since they stall every other request.

Q27.

What is the difference between the microtask queue and the macrotask (task) queue?

ADVANCED

Macrotasks include timers, I/O callbacks, and `setImmediate` — each phase of the event loop processes one category of macrotask. **Microtasks** — Promise callbacks and `process.nextTick` — are drained completely (including any new microtasks queued while draining) before the event loop is allowed to move to the next phase or the next macrotask. This is why chained `.then()` calls or recursive `process.nextTick()` calls can, in pathological cases, starve I/O entirely.

Q28.

How can Node.js handle CPU-intensive work without blocking the event loop?

ADVANCED

- **Worker Threads** — offload the computation to a separate thread and communicate via message passing.
- **Child processes / cluster** — spawn a separate process (possibly in another language) to do the work.
- **Break work into chunks** using `setImmediate` / `process.nextTick` to yield back to the event loop between chunks (cooperative scheduling) — useful for medium workloads.
- **Offload to an external service/queue** (e.g. a job queue processed by dedicated workers) for genuinely heavy work like video transcoding.

4 Streams & Buffers

How Node efficiently handles large amounts of data.

Q29. What are Streams in Node.js? Name the four types.

INTERMEDIATE

Streams are objects that let you read or write data **incrementally**, in chunks, instead of loading it all into memory at once — essential for handling large files, network data, or continuous data sources efficiently.

- **Readable** — a source you can read from (e.g. `fs.createReadStream`).
- **Writable** — a destination you can write to (e.g. `fs.createWriteStream`).
- **Duplex** — both readable and writable (e.g. a TCP socket).
- **Transform** — a duplex stream that modifies data as it passes through (e.g. `zlib.createGzip`).

Q30. What is backpressure, and how do streams handle it?

ADVANCED

Backpressure happens when a writable destination can't consume data as fast as a readable source produces it (e.g. reading from a fast SSD but writing over a slow network). Without handling this, unread data would pile up in memory and could crash the process. Node's streams handle it automatically:

`writable.write(chunk)` returns `false` when its internal buffer is full, signaling the source to pause; once the buffer drains, the writable emits a `'drain'` event telling the source it's safe to resume. `.pipe()` implements this dance for you automatically.

Q31. What does the pipe() method do internally? INTERMEDIATE

`readable.pipe(writable)` connects a readable stream's output directly to a writable stream's input. Internally it listens for `'data'` events on the readable, calls `write()` on the writable for each chunk, automatically pauses the readable when the writable signals backpressure, resumes it on `'drain'`, and ends the writable when the readable emits `'end'`. This avoids manually wiring all of that plumbing (and its error-prone backpressure handling) yourself.

JAVASCRIPT

```
const fs = require('fs');
const zlib = require('zlib');

fs.createReadStream('input.txt')
  .pipe(zlib.createGzip())
  .pipe(fs.createWriteStream('input.txt.gz'))
  .on('finish', () => console.log('Compression complete'));
```

Q32.**What is a Buffer, and why does Node need it separately from strings?**INTERMEDIATE

A `Buffer` is a fixed-length chunk of raw binary data allocated outside the V8 JS heap. JavaScript strings are UTF-16 encoded and not well-suited for handling arbitrary binary data (like image bytes, TCP packets, or file contents that aren't valid text). Buffers let Node work efficiently with binary data from files, sockets, and streams without unnecessary encoding/decoding overhead.

Q33. Difference between Buffer, TypedArray, and ArrayBuffer.

ADVANCED

ArrayBuffer is a raw, fixed-length block of memory with no way to directly read/write it. A **TypedArray** (e.g. **Uint8Array**) is a "view" that lets you interpret an ArrayBuffer's bytes as a specific numeric type. Node's **Buffer** is a subclass of **Uint8Array** with extra Node-specific convenience methods (like `.toString('base64')` or `.write()`), making it interoperable with standard JS typed arrays while adding Node ergonomics.

Q34. How would you efficiently read a very large file in Node.js?

INTERMEDIATE

Use a **stream** (`fs.createReadStream`) instead of `fs.readFile` / `readFileSync`, which load the entire file into memory. Streams read the file in configurable chunks, process each chunk as it arrives, and keep memory usage constant regardless of file size — critical for files larger than available RAM.

JAVASCRIPT

```
const fs = require('fs');

const stream = fs.createReadStream('huge-log.txt', {
  encoding: 'utf8',
  highWaterMark: 64 * 1024, // 64KB chunks
});

let lineCount = 0;
stream.on('data', (chunk) => {
  lineCount += chunk.split('\n').length - 1;
});
stream.on('end', () => console.log(`Total lines: ${lineCount}`));
stream.on('error', (err) => console.error('Read failed:', err));
```

Q35.

Implement a simple Transform stream (e.g. uppercase text as it flows through).

ADVANCED**JAVASCRIPT**

```
const { Transform } = require('stream');

class UppercaseTransform extends Transform {
  _transform(chunk, encoding, callback) {
    this.push(chunk.toString().toUpperCase());
    callback(); // signal this chunk is done
  }
}

process.stdin
  .pipe(new UppercaseTransform())
  .pipe(process.stdout);
```

Q36. What key events do Readable and Writable streams emit?**INTERMEDIATE**

- **'data'** — emitted (Readable) when a chunk is available (switches the stream to flowing mode).
- **'end'** — emitted (Readable) when there is no more data to consume.
- **'drain'** — emitted (Writable) when it's safe to resume writing after backpressure.
- **'finish'** — emitted (Writable) when **.end()** has been called and all data has been flushed.
- **'error'** — emitted by either when something goes wrong; unhandled stream errors crash the process, so always attach a handler.
- **'close'** — emitted when the underlying resource has been closed.

5 File System & OS

Q37.

Difference between `fs.readFile`, `fs.readFileSync`, and `fs.createReadStream`.

BEGINNER

`readFileSync` blocks and returns the full file content. `readFile` is async (callback/promise-based) but still loads the **entire** file into memory at once before returning it. `createReadStream` reads incrementally in chunks and never holds the whole file in memory — the right choice for large files or when you want to start processing before the whole file has arrived.

Q38. How do you watch for file changes in Node.js? INTERMEDIATE

The built-in `fs.watch(path, callback)` uses OS-native file system events (inotify on Linux, FSEvents on macOS, ReadDirectoryChangesW on Windows). It's fast but has known cross-platform inconsistencies (e.g. duplicate events, unreliable filenames on some platforms). For production tooling (like dev-server file watchers), most projects use **chokidar**, which wraps `fs.watch` /polling and normalizes behavior across platforms.

Q39. What is the `os` module used for? BEGINNER

The `os` module exposes operating-system-level information: `os.cpus()` (core count/info, used to size a cluster), `os.freemem()` / `os.totalmem()`, `os.platform()`, `os.homedir()`, `os.tmpdir()`, and network interface details.

Q40.**Why should you use the path module instead of manually concatenating strings for file paths?****BEGINNER**

Path separators differ across operating systems (`/` on POSIX, `\` on Windows). The `path` module (`path.join`, `path.resolve`, `path.extname`, etc.) builds correct, normalized paths for whichever OS the code runs on, and safely handles edge cases like trailing slashes or `..` segments — hand-built strings break silently across platforms.

Q41. Difference between `__dirname` and `process.cwd()`.**INTERMEDIATE**

`__dirname` is the absolute path of the directory containing the **currently executing file** — it never changes regardless of where the process was launched from. `process.cwd()` is the **current working directory of the process** — the directory the terminal was in when `node` was invoked (and can even change at runtime via `process.chdir()`). For file paths relative to your source code, use `__dirname`; for paths relative to how the user invoked the program, use `process.cwd()`.

6 Error Handling

Q42.

How do you handle errors differently in synchronous vs asynchronous code?

BEGINNER

Synchronous errors are caught with `try/catch` around the call site. Traditional callback-based async code cannot be caught by a wrapping `try/catch` (the callback runs later, outside that call stack) — it instead follows the **error-first callback** convention. Promise-based code uses `.catch()`, and `async` / `await` code can use `try/catch` again because `await` re-throws rejections into the current function's context.

Q43. What is the error-first callback pattern?

BEGINNER

A Node.js convention where every async callback's first parameter is reserved for an error (`null` if none occurred), and subsequent parameters carry the result. It gives callback-based APIs a consistent, predictable error-handling contract across the entire ecosystem.

JAVASCRIPT

```
fs.readFile('config.json', 'utf8', (err, data) => {
  if (err) {
    return console.error('Failed to read config:', err.message);
  }
  console.log(JSON.parse(data));
});
```


Q44.

How do unhandled promise rejections behave, and how should you deal with them?

INTERMEDIATE

If a Promise rejects and nothing calls `.catch()` on it (or awaits it inside a `try/catch`), Node emits an `'unhandledRejection'` event on `process`. In modern Node, unhandled rejections terminate the process by default (matching uncaught exception behavior), since an unhandled error usually means the app is in an unknown/unsafe state. Best practice is to always attach `.catch()` / `try-catch` at the boundaries where promises are created or consumed, rather than relying on the global handler as your only safety net.

Q45.

Should you use `process.on('uncaughtException')` / `process.on('unhandledRejection')` in production?

ADVANCED

They're useful as a **last-resort safety net** — logging the error for diagnostics before shutting down gracefully — but not as a way to "swallow" errors and keep running. After an uncaught exception, the process may be in a corrupted state (partially-updated data structures, leaked resources), so the recommended pattern is: log the error, alert/report it, close servers and connections cleanly, then `process.exit(1)` and let a process manager (PM2, Kubernetes, systemd) restart the process fresh.

Q46.**What's the difference between operational errors and programmer errors?****ADVANCED**

Operational errors are runtime problems in a correctly-written program: failed DB connection, invalid user input, network timeout, file not found. These are expected and should be handled gracefully (retry, return a 4xx response, log and continue).

Programmer errors are bugs: calling a function with the wrong type, undefined is not a function, broken logic. These indicate an invalid program state and generally should not be "handled" — they should crash loudly (in dev) or trigger a controlled restart (in prod), since limping along risks worse, silent corruption.

Q47. How and why would you create custom error classes?

INTERMEDIATE

Custom error classes (extending the built-in `Error`) let you attach structured metadata (status codes, error codes, whether it's operational) and let calling code distinguish error types with `instanceof` instead of parsing message strings — much more robust for centralized error-handling middleware.

JAVASCRIPT

```
class AppError extends Error {
  constructor(message, statusCode = 500) {
    super(message);
    this.name = this.constructor.name;
    this.statusCode = statusCode;
    this.isOperational = true;
    Error.captureStackTrace(this, this.constructor);
  }
}

class NotFoundError extends AppError {
  constructor(resource) {
    super(`${resource} not found`, 404);
  }
}

// usage
throw new NotFoundError('User');
```

7 Process, Child Processes, Cluster & Worker Threads

Q48. What is the process object, and what are its key members?

BEGINNER

- `process.env` — environment variables.
- `process.argv` — command-line arguments.
- `process.exit(code)` — terminates the process.
- `process.on(event, handler)` — listen for signals/lifecycle events (`'exit'`, `'SIGTERM'`, `'uncaughtException'`).
- `process.pid`, `process.platform`, `process.version` — runtime metadata.
- `process.memoryUsage()`, `process.cpuUsage()` — resource diagnostics.

Q49.

child_process: differences between exec, execFile, spawn, and fork.

ADVANCED

- `exec(command)` — runs a command in a shell, buffers all stdout/stderr in memory, returns it in a callback. Good for short commands with small output; risky with untrusted input (shell injection) since it goes through `/bin/sh`.
- `execFile(file, args)` — runs an executable directly without a shell, safer against injection, still buffers output.
- `spawn(command, args)` — launches a process and streams stdout/stderr, ideal for long-running processes or large output since nothing is buffered in memory at once.
- `fork(modulePath)` — a specialized `spawn` just for launching another **Node.js** module as a separate process, with a built-in IPC channel for message passing (`.send()` / `'message'`).

Q50.**What is the cluster module, and how does it help scale a Node app across CPU cores?****ADVANCED**

Since one Node process runs JS on a single thread, it can only use one CPU core by default. The `cluster` module lets you fork multiple copies of your process (typically one per CPU core) that all share the same server port — the OS/master process load-balances incoming connections across them. Each worker is a fully separate process with its own memory and event loop, so a crash in one worker doesn't take down the others.

JAVASCRIPT

```
const cluster = require('cluster');
const os = require('os');
const http = require('http');

if (cluster.isPrimary) {
  const cpuCount = os.cpus().length;
  for (let i = 0; i < cpuCount; i++) cluster.fork();

  cluster.on('exit', (worker) => {
    console.log(`Worker ${worker.process.pid} died, restarting...`);
    cluster.fork();
  });
} else {
  http.createServer((req, res) => res.end('Handled by worker ' + process.pid))
    .listen(3000);
}
```

Q51.**Worker Threads vs Cluster vs Child Process — when do you use each?****ADVANCED**

- **Cluster** — scaling a network server (HTTP API) across CPU cores; each worker is an independent process sharing a listening port.
- **Worker Threads** — offloading CPU-intensive JavaScript computation (image processing, parsing, hashing) within the same process, with lighter-weight threads that can share memory via `SharedArrayBuffer`.
- **Child Process** — running an entirely separate program (a shell command, a Python script, a different executable) or isolating untrusted/crash-prone work in its own OS process.

Q52.**How does Node's cluster module load-balance requests across workers?****ADVANCED**

On most platforms (not Windows), the default strategy is **round-robin** (`SCHED_RR`): the primary process listens on the socket and distributes incoming connections to workers in turn. Alternatively, `SCHED_NONE` leaves distribution to the OS kernel, which typically sends a new connection to whichever worker's listener happens to accept it first (can result in uneven load in practice).

Q53. What is PM2 and why use it in production? **INTERMEDIATE**

PM2 is a production process manager for Node.js applications. It provides: automatic restarts on crash, a built-in cluster mode (spawning multiple instances without hand-writing cluster code), zero-downtime reloads, log management, and monitoring (CPU/memory per process). It removes the need to hand-roll process supervision, though in containerized/Kubernetes deployments that responsibility is often delegated to the orchestrator instead.

Q54. How do you gracefully shut down a Node.js HTTP server?

ADVANCED

Listen for termination signals (`SIGTERM` from orchestrators like Kubernetes/Docker, `SIGINT` from Ctrl+C), stop accepting new connections, let in-flight requests finish, close DB connections/queues, then exit. This avoids dropping active requests during deploys or restarts.

JAVASCRIPT

```
const server = app.listen(3000);

function shutdown(signal) {
  console.log(`${signal} received, shutting down gracefully...`);
  server.close(async () => {
    await db.close();
    console.log('Cleanup complete, exiting.');
    process.exit(0);
  });
  // force-exit if shutdown hangs beyond a timeout
  setTimeout(() => process.exit(1), 10_000).unref();
}

process.on('SIGTERM', () => shutdown('SIGTERM'));
process.on('SIGINT', () => shutdown('SIGINT'));
```

8 Express.js, Middleware & REST APIs

Q55.

What is Express.js and why use it instead of the raw http module?

BEGINNER

Express is a minimal, unopinionated web framework built on top of Node's core `http` module. Raw `http` requires manually parsing URLs, routing, request bodies, and handling response formatting. Express provides routing, a middleware pipeline, request/response helper methods, and a large ecosystem of plugins, dramatically reducing boilerplate for building APIs and web servers.

Q56. What is middleware in Express? Explain the req/res/next flow.

BEGINNER

Middleware is a function with the signature `(req, res, next)` that sits in the request-processing pipeline. It can inspect/modify the request or response, end the request-response cycle, or call `next()` to pass control to the next middleware/route handler. Middleware runs in the order it's registered, forming a chain.

JAVASCRIPT

```
const express = require('express');
const app = express();

function requestLogger(req, res, next) {
  console.log(`${req.method} ${req.url}`);
  next(); // pass control to the next handler
}

app.use(requestLogger);
app.use(express.json()); // built-in body-parsing middleware

app.get('/health', (req, res) => res.json({ status: 'ok' }));
app.listen(3000);
```


Q57. What types of middleware exist in Express?

INTERMEDIATE

- **Application-level** — bound via `app.use()` / `app.METHOD()`, runs for every matching request.
- **Router-level** — same idea, bound to an `express.Router()` instance for modular route groups.
- **Built-in** — e.g. `express.json()`, `express.static()`.
- **Third-party** — e.g. `helmet`, `cors`, `morgan`.
- **Error-handling** — a special middleware with 4 parameters, `(err, req, res, next)`, that Express routes errors to.

Q58.**How does error-handling middleware differ from regular middleware?****INTERMEDIATE**

Express recognizes error-handling middleware specifically because it declares **four** parameters: `(err, req, res, next)`. Calling `next(err)` anywhere in the chain skips all remaining regular middleware and jumps straight to the nearest error handler. It must be registered **last**, after all routes, and centralizes error formatting/logging instead of repeating try/catch response logic in every route.

JAVASCRIPT

```
app.get('/users/:id', async (req, res, next) => {
  try {
    const user = await userService.findById(req.params.id);
    if (!user) throw new NotFoundError('User');
    res.json(user);
  } catch (err) {
    next(err); // forward to the error-handling middleware
  }
});

// Must be registered after all routes
app.use((err, req, res, next) => {
  const status = err.statusCode || 500;
  res.status(status).json({ error: err.message });
});
```

Q59. How would you structure a scalable Express/Node REST API?

ADVANCED

A common, testable layout separates concerns into layers:

- **Routes** — define URL paths and HTTP verbs, delegate to controllers.
- **Controllers** — parse/validate request data, call services, shape the HTTP response.
- **Services** — business logic, independent of HTTP (easy to unit test and reuse).
- **Repositories / data access layer** — database queries, isolated from business logic.
- **Middleware** — cross-cutting concerns (auth, validation, logging, error handling).
- **Config** — environment-driven settings, kept out of business logic.

This keeps each layer independently testable and makes it easy to, say, swap the database or expose the same services over gRPC later.

Q60. What is CORS, and how do you enable it in a Node.js API?

INTERMEDIATE

CORS (Cross-Origin Resource Sharing) is a browser security mechanism that blocks a web page from making requests to a different origin (domain/port/protocol) unless the server explicitly allows it via response headers (`Access-Control-Allow-Origin`, etc.). It's a browser-enforced restriction, not a server-side one — servers just declare who's allowed. In Express, it's typically handled with the `cors` package.

JAVASCRIPT

```
const cors = require('cors');

app.use(cors({
  origin: ['https://myapp.com'],
  methods: ['GET', 'POST', 'PUT', 'DELETE'],
  credentials: true,
}));
```

Q61. How do you validate incoming request data in a Node API?

INTERMEDIATE

Never trust client input. Use a schema-validation library — **Zod**, **Joi**, or **express-validator** — to validate and coerce request bodies/params/query strings against a defined schema before they reach business logic, rejecting malformed requests early with a clear 400 error.

JAVASCRIPT

```
const { z } = require('zod');

const createUserSchema = z.object({
  email: z.string().email(),
  age: z.number().int().min(0).max(130),
});

app.post('/users', (req, res, next) => {
  const result = createUserSchema.safeParse(req.body);
  if (!result.success) {
    return res.status(400).json({ errors: result.error.flatten() });
  }
  // result.data is now validated and typed
  next();
});
```

Q62.

What is REST, and what does "idempotent" mean for HTTP verbs?

INTERMEDIATE

REST (Representational State Transfer) is an architectural style for designing networked APIs around stateless requests, resources identified by URLs, and standard HTTP verbs/status codes. An operation is **idempotent** if performing it multiple times has the same effect as performing it once: **GET**, **PUT**, and **DELETE** are idempotent (repeating a **DELETE** on an already-deleted resource still leaves it deleted); **POST** is **not** idempotent (submitting it twice typically creates two resources); **PATCH** may or may not be, depending on implementation.

Q63. How would you implement pagination in an API? INTERMEDIATE

Two common approaches:

- **Offset-based** (`?page=2&limit=20` , translated to SQL `LIMIT/OFFSET`) — simple, but slow/inconsistent on large tables if rows are inserted/deleted between requests.
- **Cursor-based** (`?after=lastId&limit=20` , using a `WHERE id > lastId` query) — more consistent under concurrent writes and scales better on large datasets, at the cost of not supporting arbitrary "jump to page N" navigation.

Responses should include metadata (total count or `hasNextPage` , next cursor) so clients know how to fetch the following page.

Q64. How would you implement rate limiting in an Express app?INTERMEDIATE

Track request counts per client (by IP, API key, or user ID) over a rolling time window, and reject requests over the limit with a `429 Too Many Requests` . For a single instance, an in-memory counter (e.g. `express-rate-limit`) works; for multiple instances behind a load balancer, the counter needs to live in a shared store like **Redis** so all instances agree on the count.

JAVASCRIPT

```
const rateLimit = require('express-rate-limit');

const limiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 100,                  // 100 requests per window per IP
  standardHeaders: true,
  message: { error: 'Too many requests, please try again later.' },
});

app.use('/api/', limiter);
```

9 Security & Authentication

Q65. What are the top Node.js/Express security best practices?

ADVANCED

- Use **Helmet** to set protective HTTP headers.
- Validate and sanitize **all** user input; never build queries via string concatenation.
- Never use `eval()` or the `Function` constructor on untrusted input.
- Keep dependencies updated and audit them (`npm audit`) for known CVEs.
- Store secrets in environment variables/secret managers, never in source control.
- Use HTTPS/TLS everywhere; set `secure` and `httpOnly` flags on cookies.
- Rate-limit auth-sensitive endpoints and apply proper CORS restrictions.
- Hash passwords with bcrypt/argon2, never store or log them in plaintext.

Q66.

How does JWT authentication work? What's the point of refresh tokens?

INTERMEDIATE

On login, the server verifies credentials and issues a signed JSON Web Token containing claims (user ID, roles, expiry) that the client stores and sends with future requests (usually as an `Authorization: Bearer` header). The server verifies the signature on each request — no session lookup needed, which is what makes JWTs stateless and scalable.

Access tokens are typically short-lived (minutes) to limit damage if leaked. A longer-lived **refresh token**, stored more securely (`httpOnly` cookie), is used only to obtain a new access token when the old one expires, without forcing the user to log in again — and can be revoked server-side if compromised, unlike a stateless access token.

Q67. Session-based vs token-based authentication — tradeoffs.

INTERMEDIATE

- **Session-based** — server stores session state (in memory, Redis, DB) keyed by a session ID stored in a cookie. Easy to revoke instantly; requires a shared session store across multiple server instances.
- **Token-based (JWT)** — stateless, self-contained, scales horizontally without a shared store; harder to revoke before expiry (needs a blocklist or short expiry + refresh tokens), and the token payload is readable (though not modifiable) by anyone who has it.

Q68. How should passwords be stored securely?

INTERMEDIATE

Never store plaintext or reversibly-encrypted passwords. Use a slow, purpose-built **hashing** algorithm — **bcrypt** or **argon2** — which automatically incorporates a random **salt** per password (defeating rainbow-table attacks) and is deliberately slow/tunable to resist brute-forcing, unlike fast general-purpose hashes like MD5/SHA-256.

JAVASCRIPT

```
const bcrypt = require('bcrypt');

async function hashPassword(plain) {
  const saltRounds = 12;
  return bcrypt.hash(plain, saltRounds);
}

async function verifyPassword(plain, hash) {
  return bcrypt.compare(plain, hash);
}
```

Q69. What is CSRF, and how do you prevent it? ADVANCED

Cross-Site Request Forgery tricks a logged-in user's browser into submitting an unwanted request to your site (e.g. via a malicious form on another page), exploiting the fact that cookies are sent automatically. It mainly affects cookie-based session auth. Prevention: CSRF tokens (a unique, unpredictable token embedded in forms/headers and verified server-side), the `SameSite=Strict/Lax` cookie attribute, and checking the `Origin` / `Referer` header. Pure token-based (Bearer header) APIs are inherently less exposed since tokens aren't sent automatically by the browser.

Q70. How do you prevent SQL/NoSQL injection in Node.js?INTERMEDIATE

- Always use **parameterized queries / prepared statements** (or an ORM/query builder that does this for you) instead of concatenating user input into query strings.
- For MongoDB, disable/sanitize operator injection (e.g. a request body like `{"$gt": ""}` being passed straight into a query) using libraries like `express-mongo-sanitize`, and validate input types strictly.
- Apply the principle of least privilege to the DB user your app connects as.

JAVASCRIPT

```
// Vulnerable: string concatenation
db.query(`SELECT * FROM users WHERE email = '${email}'`);

// Safe: parameterized query - the driver handles escaping
db.query('SELECT * FROM users WHERE email = ?', [email]);
```


Q71.**How should you handle secrets and environment variables in a Node.js app?****BEGINNER**

Keep secrets (API keys, DB credentials, JWT signing keys) out of source control entirely. Load them from environment variables (via `process.env`, often with the `dotenv` package for local development), and in production use a proper secret manager (AWS Secrets Manager, HashiCorp Vault, Doppler, or the platform's built-in secret store) rather than plain `.env` files on disk. Always add `.env` to `.gitignore`.

Q72. What is Helmet.js and what does it protect against?**INTERMEDIATE**

Helmet is Express middleware that sets a collection of security-related HTTP response headers with sane defaults, protecting against several common attack classes with a single line: `Content-Security-Policy` (XSS/data-injection), `Strict-Transport-Security` (forces HTTPS), `X-Frame-Options` (clickjacking), `X-Content-Type-Options` (MIME-sniffing attacks), and hides the `X-Powered-By` header so attackers can't easily fingerprint the stack.

JAVASCRIPT

```
const helmet = require('helmet');  
app.use(helmet());
```

10 Database Integration & ORMs

Q73. How do you connect Node.js to MongoDB vs a SQL database?

BEGINNER

For MongoDB, the common choice is **Mongoose**, an ODM (Object-Document Mapper) that adds schemas, validation, and model methods on top of the official MongoDB driver. For SQL databases (PostgreSQL, MySQL), common choices are **Prisma** or **Sequelize** (full ORMs with migrations and query builders) or lighter query builders like **Knex**. All of them manage a connection pool under the hood rather than opening a new connection per query.

Q74. What is connection pooling, and why does it matter?

INTERMEDIATE

Opening a new database connection per request is slow (TCP handshake, auth negotiation) and can quickly exhaust the database's max-connections limit under load. A connection pool keeps a set of already-established connections ready to reuse: a query borrows a free connection, runs, and returns it to the pool instead of closing it. This dramatically improves throughput and protects the database from being overwhelmed by connection churn.

Q75. ORM vs ODM vs raw query builder — what are the tradeoffs?

INTERMEDIATE

- **ORM/ODM** (Prisma, Sequelize, Mongoose, TypeORM) — map database records to JS objects/models, provide validation, relations, and migrations. Faster to develop with and safer by default (parameterized queries built-in), but can generate inefficient queries if used carelessly (see N+1, Q77) and add an abstraction layer to reason about.
- **Query builder** (Knex, Drizzle) — a thinner layer that constructs SQL programmatically without full object mapping — more control over the exact query, less "magic."
- **Raw SQL** — maximum control and performance, but you own all the parameterization/escaping and portability yourself.

Q76. How do you handle database transactions in Node.js?

ADVANCED

A transaction groups multiple operations so they either all succeed (commit) or all fail together (rollback) — essential when multiple writes must stay consistent (e.g. debit one account, credit another). Most ORMs expose a transaction API that wraps a callback or a set of operations.

JAVASCRIPT

```
// Example with Prisma
await prisma.$transaction(async (tx) => {
  await tx.account.update({
    where: { id: fromId },
    data: { balance: { decrement: amount } },
  });
  await tx.account.update({
    where: { id: toId },
    data: { balance: { increment: amount } },
  });
  // If either update throws, the whole transaction rolls back automat
});
```

Q77. What is the N+1 query problem, and how do you avoid it?

ADVANCED

It happens when you fetch a list of N records, then run a separate query for each one to fetch related data — resulting in 1 + N total queries instead of a couple. It's easy to introduce accidentally with ORMs (e.g. looping over users and calling `user.getPosts()` for each). Fix by **eager loading**/joining the related data upfront (e.g. Prisma's `include`, Sequelize's `include`, or a single SQL `JOIN`) or using a batching utility like **DataLoader** to collapse many individual lookups into one batched query.

Q78.**What is database indexing, and why does it matter for a Node backend?****INTERMEDIATE**

An index is a data structure (usually a B-tree) that lets the database find rows matching a condition without scanning the entire table. Without an index on a frequently-queried column (e.g. `email`, a foreign key used in joins), a query degrades to a full table scan — fine at small scale, but a major bottleneck as data grows, often the actual root cause behind a Node API that "feels slow" even though the application code is correct.

11 Testing & Debugging

Q79. How do you write unit tests for a Node.js app? BEGINNER

Common frameworks: **Jest** (test runner + assertions + mocking built in) or **Mocha** paired with an assertion library like **Chai**. Tests isolate a single unit of logic (a function, a class method), asserting its output for given inputs, independent of I/O.

JAVASCRIPT

```
// sum.js
function sum(a, b) { return a + b; }
module.exports = { sum };

// sum.test.js
const { sum } = require('./sum');

describe('sum()', () => {
  it('adds two positive numbers', () => {
    expect(sum(2, 3)).toBe(5);
  });

  it('handles negative numbers', () => {
    expect(sum(-1, -1)).toBe(-2);
  });
});
```

Q80. What is mocking/stubbing, and why is it used in tests?

INTERMEDIATE

Mocking replaces a real dependency (a database call, an external API, the file system) with a fake, controllable version so tests are fast, deterministic, and don't depend on external systems being available. Jest provides

`jest.mock()` / `jest.fn()` built in; **Sinon** is a popular standalone library for spies, stubs, and mocks when using Mocha.

JAVASCRIPT

```
jest.mock('../db');
const db = require('../db');

test('getUser returns formatted user', async () => {
  db.findUser.mockResolvedValue({ id: 1, name: 'Ada' });

  const result = await getUser(1);

  expect(db.findUser).toHaveBeenCalledWith(1);
  expect(result.name).toBe('Ada');
});
```

Q81.

Difference between unit, integration, and end-to-end (e2e) tests.

INTERMEDIATE

- **Unit tests** — test a single function/module in isolation, dependencies mocked. Fast, cheap, pinpoint failures precisely.
- **Integration tests** — test how multiple units work together (e.g. a route handler + a real test database), catching issues that mocks would hide.
- **E2E tests** — test the full running system as a user/client would (hitting real HTTP endpoints, sometimes through a browser). Slowest and most brittle, but highest confidence the whole system actually works together.

A healthy test suite has many unit tests, a moderate number of integration tests, and few, high-value e2e tests (the "testing pyramid").

Q82. How do you debug a running Node.js application?

INTERMEDIATE

- `node --inspect app.js` (or `--inspect-brk` to pause on the first line) exposes a debugging protocol you can attach Chrome DevTools or VS Code's debugger to — full breakpoints, call stack inspection, watch expressions.
- The `debugger;` statement pauses execution at that line when running under an inspector.
- `console.log` / `console.table` / `console.trace` for quick, low-effort inspection.
- For production issues, structured logging plus tools like Node's built-in profiler or APM products (Datadog, New Relic) are typically used instead of live-attaching a debugger.

Q83. What is test coverage, and how do you measure it? BEGINNER

Test coverage measures how much of the codebase is exercised by the test suite — commonly reported as line, branch, function, and statement percentages. Jest has coverage built in (`jest --coverage`, powered by Istanbul/nyc). High coverage doesn't guarantee correctness (tests can execute code without meaningfully asserting on it), but low coverage reliably signals under-tested areas.

Q84. How do you test HTTP API endpoints in Node.js? INTERMEDIATE

Supertest is the standard tool — it lets you make requests directly against your Express app object in-process (no need to actually bind a port), and assert on status codes, headers, and response bodies.

JAVASCRIPT

```
const request = require('supertest');
const app = require('../app');

describe('GET /users/:id', () => {
  it('returns 404 for an unknown user', async () => {
    const res = await request(app).get('/users/999');
    expect(res.status).toBe(404);
  });

  it('returns the user for a valid id', async () => {
    const res = await request(app).get('/users/1');
    expect(res.status).toBe(200);
    expect(res.body).toHaveProperty('id', 1);
  });
});
```


12 Performance, Memory & Scaling

Q85. How do you identify and fix memory leaks in a Node.js app?

ADVANCED

Symptoms: steadily climbing memory usage over time (visible via `process.memoryUsage()` or infra monitoring) that never returns to baseline, eventually crashing with an out-of-memory error. Diagnosis approach: take heap snapshots at intervals (via `--inspect` and Chrome DevTools' Memory tab, or `clinic.js` / `heapdump`), compare them to see which object types keep growing, then trace back to what's still referencing them (retained-size / "who's holding this" analysis in DevTools) and remove the unwanted reference.

Q86.

What are common causes of memory leaks in Node applications?

ADVANCED

- **Global variables/caches** that grow unbounded (e.g. an in-memory cache with no eviction policy or size limit).
- **Forgotten event listeners** — repeatedly calling `emitter.on()` without ever removing them (especially in a loop or on reused objects) keeps referenced closures alive forever.
- **Uncleared timers** — a `setInterval` that's never `clearInterval`'d keeps its closure (and everything it references) alive.
- **Closures** unintentionally capturing large objects they don't need.
- Accumulating **detached objects** (like large arrays pushed to but never trimmed) tracked in module-level state, which lives for the life of the process.

Q87.**How do you profile a Node.js application's CPU/memory usage?****ADVANCED**

- `node --prof app.js` generates a V8 tick log you can process with `node --prof-process` to see where CPU time goes.
- Chrome DevTools' Performance/Memory tabs, attached via `--inspect`, give flame graphs and heap snapshots interactively.
- **clinic.js** (Doctor, Flame, Bubbleprof) wraps profiling into a single command and produces readable visual reports, great for diagnosing event-loop delay, CPU bottlenecks, and I/O bottlenecks specifically.
- APM tools (Datadog, New Relic, Prometheus + Grafana) for continuous production-level profiling and alerting.

Q88. Vertical vs horizontal scaling for a Node.js application.**INTERMEDIATE**

Vertical scaling — give the single instance more resources (CPU, RAM). Since one Node process is limited to one core for JS execution, vertical scaling alone hits diminishing returns unless paired with clustering/worker threads to actually use the extra cores. **Horizontal scaling** — run more instances (processes, containers, or machines) behind a load balancer. This is the more common approach for Node in production, since it also improves fault tolerance (one instance crashing doesn't take down the whole service).

Q89. How does garbage collection work in V8?**ADVANCED**

V8 uses a **generational** garbage collector based on the observation that most objects die young. The heap is split into a small **young generation** (new space), collected frequently and cheaply via a fast copying collector ("Scavenger"), and an **old generation**, for objects that survive multiple young-generation collections, collected less often using a mark-sweep-compact algorithm since scanning it is more expensive. This generational split keeps most GC pauses very short, though large old-generation collections can still cause noticeable pauses if the heap grows large.

Q90.**What caching strategies are commonly used in Node.js applications?****INTERMEDIATE**

- **In-memory cache** (a plain `Map`, or `Lru-cache`) — fastest, but local to one process and lost on restart; doesn't work across multiple instances without going stale relative to each other.
- **Redis/Memcached** — a shared cache all instances can read/write, ideal for horizontally-scaled deployments; supports TTL-based expiry.
- **HTTP caching headers** (`Cache-Control`, `ETag`) — let clients/CDNs/proxies cache responses without hitting your server at all.
- **CDN caching** — for static assets or cacheable API responses served at the edge, closest to the user.

Q91. How do you design a Node.js API to handle high throughput?**ADVANCED**

- Use connection pooling for databases to avoid connection overhead per request.
- Scale horizontally with clustering/multiple instances behind a load balancer.
- Cache expensive/frequently-read data (Redis) to reduce database load.
- Offload slow or bursty work (emails, image processing, report generation) to a background job queue (BullMQ, RabbitMQ) instead of doing it inline in the request.
- Use streaming responses for large payloads instead of buffering everything in memory.
- Add rate limiting so a few abusive clients can't degrade service for everyone else.

Q92.**What happens when the event loop is blocked, and how do you detect it?****ADVANCED**

If a synchronous operation takes too long (a heavy computation, a large synchronous `JSON.parse`, a poorly-chosen `*Sync` fs call), the single JS thread can't process **any** other request, timer, or I/O callback until it finishes — every concurrent client stalls simultaneously, which is far worse than one slow request in a thread-per-request model. Detection: monitor **event loop lag** (the delay between when a `setImmediate`/timer is scheduled and when it actually fires) using a library like `toobusy-js` or Node's built-in `perf_hooks` event loop monitor, and alert when lag crosses a threshold.

13 Microservices & Real-time

Q93.

Monolith vs microservices — when does Node.js fit well for microservices?

INTERMEDIATE

A monolith is one deployable application containing all functionality; microservices split functionality into independently deployable services communicating over the network. Node.js suits microservices well because it starts fast, has a small footprint, and handles I/O-bound inter-service communication efficiently — good for lightweight API gateways, BFFs (backend-for-frontend), and services that are mostly I/O orchestration rather than heavy computation. The tradeoff is added operational complexity: network calls, distributed tracing, and data consistency across services.

Q94. How do microservices typically communicate with each other?

INTERMEDIATE

- **Synchronous** — REST over HTTP (simple, widely supported) or **gRPC** (binary protocol over HTTP/2, strongly-typed contracts via Protocol Buffers, lower latency, ideal for internal service-to-service calls).
- **Asynchronous** — message queues/event streams (RabbitMQ, Kafka, SQS), where a service publishes an event and one or more consumers react independently, decoupling services in time and improving resilience to a downstream service being temporarily unavailable.

Q95.**What is a message queue, and when would you reach for RabbitMQ or Kafka in a Node.js system?****ADVANCED**

A message queue decouples producers from consumers: a producer publishes a message without needing the consumer to be online or fast, and the broker holds it until a consumer processes it — useful for background jobs, retries, and smoothing out traffic spikes. **RabbitMQ** is a traditional message broker good for task queues and complex routing (exchanges/bindings). **Kafka** is a distributed event log built for high-throughput event streaming, replay, and multiple independent consumers reading the same stream at their own pace — better suited to event-sourcing/analytics-style architectures than simple job queues.

Q96.**How do you implement real-time features (WebSockets) in Node.js?****INTERMEDIATE**

The **ws** library provides a raw, lightweight WebSocket server/client. **Socket.IO** builds on top with extra conveniences: automatic reconnection, fallback transports, room/namespace support, and a simple event-based API — at the cost of needing a matching client library and a slightly non-standard protocol.

JAVASCRIPT

```
const { Server } = require('socket.io');
const io = new Server(3000, { cors: { origin: '*' } });

io.on('connection', (socket) => {
  console.log('Client connected:', socket.id);

  socket.on('chat:message', (msg) => {
    io.emit('chat:message', { user: socket.id, text: msg });
  });

  socket.on('disconnect', () => console.log('Client left:', socket.id));
});
```

For multiple server instances, you need a shared pub/sub backend (e.g. the Socket.IO Redis adapter) so a message from a client connected to instance A can reach a client connected to instance B.

Q97. What is the API Gateway pattern? **ADVANCED**

A single entry point that sits in front of multiple backend microservices, routing each incoming request to the appropriate service. It centralizes cross-cutting concerns — authentication, rate limiting, request logging, response aggregation from multiple services, and protocol translation — so individual services don't each need to reimplement them, and clients only need to know about one endpoint instead of every service's address.

Q98.**How do you handle authentication between services in a microservices architecture?****ADVANCED**

Common approaches: a shared, short-lived **JWT** issued by a central auth service and verified independently by each downstream service (stateless, no extra network hop per call); **mutual TLS (mTLS)** where services authenticate each other via certificates, common in service-mesh setups (Istio, Linkerd); or an internal **API key/service account** scheme for simpler systems. The API Gateway often handles end-user authentication once at the edge, then passes a verified identity/context downstream so internal services trust the gateway rather than re-authenticating the end user themselves.

14 Design Patterns & Architecture

Q99.

What design patterns commonly show up in Node.js applications?

INTERMEDIATE

- **Module pattern** — Node's file-based module system naturally encapsulates private state, exposing only what's explicitly exported.
- **Singleton** — the module cache means a plain exported instance (e.g. a DB connection) is naturally a singleton across the app.
- **Observer** — `EventEmitter`-based pub/sub within a process.
- **Middleware/Chain of Responsibility** — Express's request pipeline.
- **Factory** — functions that construct and configure objects (e.g. a function that builds a pre-configured DB client or logger).
- **Strategy** — swapping interchangeable algorithms/implementations behind a common interface (e.g. different payment providers).

Q100. What is Dependency Injection, and how is it used in Node.js?

ADVANCED

Dependency Injection means a module receives its dependencies (a DB client, a logger, another service) from the outside rather than constructing/importing them directly. This makes code far easier to unit test (you can inject a mock) and decouples modules from concrete implementations. Node doesn't need a heavy DI framework the way some languages do — often plain "constructor injection" (passing dependencies as function/constructor arguments) is enough — but frameworks like **NestJS** or libraries like **InversifyJS** provide a formal DI container for larger codebases.

JAVASCRIPT

```
// Without DI: hard to test, tightly coupled
class UserService {
  constructor() {
    this.db = require('./db'); // hardcoded dependency
  }
}

// With DI: dependency passed in, easy to mock in tests
class UserServiceDI {
  constructor(db) {
    this.db = db;
  }
  findById(id) {
    return this.db.users.findOne({ id });
  }
}

const service = new UserServiceDI(realDb); // production
const testService = new UserServiceDI(fakeDb); // tests
```

Q101. Explain the Repository pattern for data access. INTERMEDIATE

The Repository pattern puts a layer between business logic and the data source, exposing methods like `findById()` or `save()` instead of leaking raw query syntax into services. Benefits: business logic doesn't care whether data comes from Postgres, MongoDB, or an external API; it's easy to swap the underlying storage or add caching inside the repository without touching callers; and it's straightforward to mock the repository in unit tests.

Q102.**What is the 12-Factor App methodology, and how does it apply to Node.js apps?**ADVANCED

A set of best practices for building portable, scalable cloud-native applications. Points most relevant to Node.js services: store **config** in environment variables, not code; treat **backing services** (DB, cache, queue) as attached resources swappable via config; keep processes **stateless** (any state goes in a shared store like Redis/DB, not in-process memory, so instances are interchangeable); make the app **disposable** — fast startup and graceful shutdown so it works well with process managers/orchestrators; and treat **logs** as an event stream (write to stdout, let the platform handle routing/storage) rather than managing log files yourself.

Q103. Monorepo vs polyrepo for Node.js projects — tradeoffs.INTERMEDIATE

Monorepo (often using npm/pnpm/yarn workspaces, or tools like Nx/Turborepo) — all services/packages live in one repository, making it easy to share code, refactor across packages atomically, and keep versions in sync; downside is the repo grows large and CI/tooling needs to scale to it. **Polyrepo** — each service has its own repository, giving teams independent release cycles and clearer ownership boundaries, at the cost of harder cross-cutting refactors and more overhead to share common code (usually via published internal packages).

15 Practical Coding Challenges

Frequently asked "write it on the whiteboard" implementation questions.

Q104. Implement a debounce function. INTERMEDIATE

Debounce delays invoking a function until after a period of inactivity — useful for search-as-you-type inputs or resize handlers, where you only want the final call after rapid-fire events stop.

JAVASCRIPT

```
function debounce(fn, delayMs) {  
  let timeoutId;  
  return function (...args) {  
    clearTimeout(timeoutId);  
    timeoutId = setTimeout(() => fn.apply(this, args), delayMs);  
  };  
}  
  
// usage: only fires 300ms after the last call  
const debouncedSearch = debounce((query) => searchApi(query), 300);
```

Q105. Implement a throttle function. INTERMEDIATE

Throttle ensures a function runs at most once per fixed interval, no matter how often it's called — useful for scroll handlers or limiting API call frequency.

JAVASCRIPT

```
function throttle(fn, intervalMs) {  
  let lastCall = 0;  
  return function (...args) {  
    const now = Date.now();  
    if (now - lastCall >= intervalMs) {  
      lastCall = now;  
      fn.apply(this, args);  
    }  
  };  
}
```

Q106. Implement a simple EventEmitter from scratch.

ADVANCED

JAVASCRIPT

```
class SimpleEmitter {
  constructor() {
    this.listeners = new Map(); // event name -> array of callbacks
  }

  on(event, callback) {
    if (!this.listeners.has(event)) this.listeners.set(event, []);
    this.listeners.get(event).push(callback);
    return this;
  }

  off(event, callback) {
    const callbacks = this.listeners.get(event);
    if (!callbacks) return this;
    this.listeners.set(event, callbacks.filter((cb) => cb !== callback));
    return this;
  }

  emit(event, ...args) {
    const callbacks = this.listeners.get(event) || [];
    callbacks.forEach((cb) => cb(...args));
    return callbacks.length > 0;
  }

  once(event, callback) {
    const wrapper = (...args) => {
      this.off(event, wrapper);
      callback(...args);
    };
    return this.on(event, wrapper);
  }
}
```

Q107. Implement a basic LRU (Least Recently Used) cache. ADVANCED

A `Map` in JavaScript preserves insertion order, which makes it a convenient building block: re-inserting a key on access moves it to the "most recent" end, and the oldest entry is always the first key in iteration order.

JAVASCRIPT

```
class LRUCache {
  constructor(capacity) {
    this.capacity = capacity;
    this.cache = new Map();
  }

  get(key) {
    if (!this.cache.has(key)) return undefined;
    const value = this.cache.get(key);
    this.cache.delete(key);
    this.cache.set(key, value); // move to most-recently-used position
    return value;
  }

  put(key, value) {
    if (this.cache.has(key)) this.cache.delete(key);
    else if (this.cache.size >= this.capacity) {
      const oldestKey = this.cache.keys().next().value;
      this.cache.delete(oldestKey); // evict least-recently-used
    }
    this.cache.set(key, value);
  }
}
```

Q108.

Implement a promise-based retry wrapper with exponential backoff.

ADVANCED**JAVASCRIPT**

```
async function retryWithBackoff(fn, { retries = 3, baseDelayMs = 200 }  
  for (let attempt = 0; attempt <= retries; attempt++) {  
    try {  
      return await fn();  
    } catch (err) {  
      if (attempt === retries) throw err; // out of retries, propagate  
      const delay = baseDelayMs * 2 ** attempt;  
      await new Promise((resolve) => setTimeout(resolve, delay));  
    }  
  }  
}  
  
// usage  
await retryWithBackoff(() => fetchFlakyApi(), { retries: 4, baseDelayM
```


Q109.

Implement a simple token-bucket rate limiter as Express middleware.

ADVANCED

JAVASCRIPT

```
function createRateLimiter({ capacity, refillPerSecond }) {
  const buckets = new Map(); // clientId -> { tokens, lastRefill }

  return function rateLimiterMiddleware(req, res, next) {
    const clientId = req.ip;
    const now = Date.now();
    let bucket = buckets.get(clientId);

    if (!bucket) {
      bucket = { tokens: capacity, lastRefill: now };
      buckets.set(clientId, bucket);
    }

    const elapsedSeconds = (now - bucket.lastRefill) / 1000;
    bucket.tokens = Math.min(capacity, bucket.tokens + elapsedSeconds);
    bucket.lastRefill = now;

    if (bucket.tokens < 1) {
      return res.status(429).json({ error: 'Rate limit exceeded' });
    }

    bucket.tokens -= 1;
    next();
  };
}

app.use(createRateLimiter({ capacity: 10, refillPerSecond: 1 }));
```

Q110.

Read a large CSV file line by line using streams (without loading it all into memory).

ADVANCED**JAVASCRIPT**

```
const fs = require('fs');
const readline = require('readline');

async function processCsv(filePath) {
  const fileStream = fs.createReadStream(filePath);
  const rl = readline.createInterface({ input: fileStream, crlfDelay: 1000 });

  let isFirstLine = true;
  for await (const line of rl) {
    if (isFirstLine) { isFirstLine = false; continue; } // skip header
    const columns = line.split(',');
    console.log(columns);
  }
}

processCsv('large-dataset.csv');
```

Q111. Implement a deep clone function. INTERMEDIATE

`structuredClone()` (built into modern Node) handles most cases natively, including circular references and Dates/Maps/Sets. It's worth knowing how to implement one manually for interviews, though:

JAVASCRIPT

```
function deepClone(value, seen = new WeakMap()) {
  if (value === null || typeof value !== 'object') return value;
  if (seen.has(value)) return seen.get(value); // handle circular refs

  if (Array.isArray(value)) {
    const clonedArr = [];
    seen.set(value, clonedArr);
    value.forEach((item, i) => { clonedArr[i] = deepClone(item, seen); });
    return clonedArr;
  }

  const clonedObj = {};
  seen.set(value, clonedObj);
  for (const key of Object.keys(value)) {
    clonedObj[key] = deepClone(value[key], seen);
  }
  return clonedObj;
}

// Simpler, built-in alternative available in modern Node:
const cloned = structuredClone(originalObject);
```

Q112.

Write a function that limits how many async operations run concurrently (an async pool).

ADVANCED

Useful when processing a large list of tasks (e.g. 1,000 API calls) where firing them all at once with `Promise.all` would overwhelm the target server or hit rate limits.

JAVASCRIPT

```
async function asyncPool(concurrency, items, iteratorFn) {
  const results = [];
  const executing = new Set();

  for (const [index, item] of items.entries()) {
    const promise = Promise.resolve().then(() => iteratorFn(item, index));
    results.push(promise);
    executing.add(promise);

    const cleanup = () => executing.delete(promise);
    promise.then(cleanup, cleanup);

    if (executing.size >= concurrency) {
      await Promise.race(executing);
    }
  }

  return Promise.all(results);
}

// usage: at most 5 requests in flight at once
await asyncPool(5, urls, (url) => fetch(url));
```

Q113.

Build a basic HTTP server using only the core http module (no Express).

INTERMEDIATE**JAVASCRIPT**

```
const http = require('http');

const server = http.createServer((req, res) => {
  if (req.method === 'GET' && req.url === '/health') {
    res.writeHead(200, { 'Content-Type': 'application/json' });
    return res.end(JSON.stringify({ status: 'ok' }));
  }

  if (req.method === 'POST' && req.url === '/echo') {
    let body = '';
    req.on('data', (chunk) => { body += chunk; });
    req.on('end', () => {
      res.writeHead(200, { 'Content-Type': 'application/json' });
      res.end(body);
    });
    return;
  }

  res.writeHead(404, { 'Content-Type': 'application/json' });
  res.end(JSON.stringify({ error: 'Not found' }));
});

server.listen(3000, () => console.log('Server running on port 3000'));
```

Node.js Interview Questions & Answers · 113 questions across 15 topic areas